

FINM 32950: Intro to HPC in Finance

Lecture 4

Chanaka Liyanarachchi

April 7, 2025

Python

Cython

Heterogeneous Computing

CUDA

Python

Scope

- ▶ We will not introduce Python in this course.
- ▶ We will focus on CPU bound (e.g., heavy computations) applications.
- ▶ Many applications we're interested in finance belong to this category.

Execution Times and Profiling

Execution Times

- ▶ There are many ways to measure the running time of a program/function:

```
import time
```

```
start_time = time.time()
```

```
do_something()
```

```
end_time = time.time()
```

```
elapsed_time = end_time - start_time
```

- ▶ We can use `timeit` to measure times:

```
timeit print('hi')
```

- ▶ Function will be run several times; mean, std are reported.
- ▶ Useful where we're comparing functions and alternative implementations during interactive development.

Profiling

- ▶ We have several options:
 1. cProfile
 2. line-profiler
 3. ...

Example: Julia Set

- ▶ The *Julia Set* problem:
 - ▶ Refers to the computation of the Julia sets, a fractal¹ that is generated by iterating a complex mathematical function.
 - ▶ It is a computationally intensive problem that is often used to illustrate parallel programming concepts because of its *embarrassingly/naturally* parallel nature.
 - ▶ Julia Set: https://en.wikipedia.org/wiki/Julia_set
- ▶ We will use the example implementation in *High Performance Python*² with minor changes in our discussion.

¹<https://en.wikipedia.org/wiki/Fractal>

²See course references for details

Using cProfile

- ▶ Run the script command line:

```
python -m cProfile -s cumulative julia.py
```

- ▶ Python profilers:

```
https://docs.python.org/3/library/profile.html
```

- ▶ Sort by cumulative times (-s argument above) shows where most of the time is spent.

```
In [16]: prun -s cumulative julia.calc_julia(1000, 300)
Length of x: 1000
Total elements: 1000000
time taken 13.756548881530762 seconds
          36222192 function calls in 14.384 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall  filename:lineno(function)
      1   0.000   0.000   14.384   14.384 {built-in method builtins.exec}
      1   0.028   0.028   14.384   14.384 <string>:1(<module>)
      1   0.460   0.460   14.355   14.355 julia.py:23(calc_julia)
      1  10.641  10.641   13.753   13.753 julia.py:10(calculate_z)
34219980  3.112   0.000    3.112    0.000 {built-in method builtins.abs}
2002000   0.142   0.000    0.142    0.000 {method 'append' of 'list' objects}
      3   0.000   0.000    0.001    0.000 {built-in method builtins.print}
```

- ▶ Report shows:
 1. total time and total number of function calls
 2. most of the time spent in calculate_z()
 3. abs and append are used heavily and expensive

- ▶ Report helps us understand the program better:
 - ▶ important info/insights
 - ▶ spot performance and other problems
- ▶ Drawback: It doesn't show the most expensive lines in `calculate_z()`.

Line Profiler

- ▶ The line_profiler shows how much time spend by each line of code.
- ▶ Very useful tool to identify expensive lines of code in a CPU-bound program.
- ▶ To install:

```
pip install line_profiler
```

- ▶ You may need to install it on midway.
- ▶ It'll be installed in:

```
~/.local/bin/
```

- ▶ To use:

```
~/.local/bin/kernprof
```

Line profiler: Arguments

```
(base) C:\Users\chanaka\Desktop\2018HPC\Python\Greeting>kernprof --help
Usage: kernprof [-s setupfile] [-o output_file_path] scriptfile [arg] ...

Options:
  --version            show program's version number and exit
  -h, --help          show this help message and exit
  -l, --line-by-line  Use the line-by-line profiler from the line_profiler
                      module instead of Profile. Implies --builtin.
  -b, --builtin       Put 'profile' in the builtins. Use 'profile.enable()'
                      and 'profile.disable()' in your code to turn it on and
                      off, or '@profile' to decorate a single function, or
                      'with profile:' to profile a single section of code.
  -o OUTFILE, --outfile=OUTFILE
                      Save stats to <outfile>
  -s SETUP, --setup=SETUP
                      Code to execute before the code to profile
  -v, --view          View the results of the profile in addition to saving
                      it.
```

Command Line

- ▶ Use profile decorator to indicate the function you want to profile:

```
@profile  
def calculate_z(maxiter, zs, cs):
```

- ▶ To run:

```
kernprof -l -v julia.py
```

- ▶ Report shows the hotspots in `calculate_z()`:

```

Anasconda Prompt
(base) C:\Users\chanaka\Desktop\2018HPC\Python\Profiling\Julia>kernprof -l -v julial.py
Length of x: 1000
Total elements: 1000000
time taken 87.65402245521545 seconds
wrote profile results to julial.py.lprof
Timer unit: 4.10547e-07 s

Total time: 52.5717 s
File: julial.py
Function: calculate_z at line 9

Line #      Hits          Time    Per Hit   % Time  Line Contents
=====
   9
  10
  11      @profile
  12      def calculate_z(maxiter, zs, cs):
  13          """Calculate output list using Julia update rule"""
  14          output = [0] * len(zs)
  15          for i in range(len(zs)):
  16              n = 0
  17              z = zs[i]
  18              c = cs[i]
  19              while abs(z) < 2 and n < maxiter:
  20                  z = z ** z + c
  21                  n += 1
  22          output[i] = n
  23          return output

(base) C:\Users\chanaka\Desktop\2018HPC\Python\Profiling\Julia>
```

- ▶ We know the expensive lines now.
- ▶ What can we do to improve performance?

Dynamic Types

- ▶ Python is a dynamically typed language:
 - ▶ names are bound to objects at run time
 - ▶ a name can be bound to objects of different types
- ▶ Has advantages and disadvantages.
- ▶ We can write flexible code:

```
a = 4  
abs(a)
```

```
a = 3 + 4j  
abs(a)
```

- ▶ This flexibility is achieved using objects.
- ▶ Objects have data and support operations.
- ▶ Carry useful info to support various operations.

- ▶ Objects add (performance) overhead:
 - ▶ need to figure out the type at run time
 - ▶ find the appropriate operation at run time
- ▶ Some operations do not need to use objects:

`z = z * z + c`

`n += 1`

Cython

Cython

- ▶ C-Extensions for Python.
- ▶ Allow us to use static C types (to improve performance).
- ▶ Cython (<http://cython.org/>) :
 - ▶ "Superset of the Python language that additionally supports calling C functions and declaring C types on variables and class attributes"
- ▶ We use the compiler to generate efficient C code from Cython code which can be used in regular Python programs.

Using Cython

- ▶ We have to use some new steps:
 - ▶ We (as usual) write Python code in a .py file.
 - ▶ We write Cython code in a .pyx file.
 - ▶ Cython code is compiled using a C/C++ compiler (we usually use a Python script³ to compile Cython code).
 - ▶ Cython documentation:
<https://cython.readthedocs.io/en/latest/index.html>

³[https:](https://pythonhosted.org/an_example_pypi_project/setuptools.html)

[/pythonhosted.org/an_example_pypi_project/setuptools.html](https://pythonhosted.org/an_example_pypi_project/setuptools.html)

Example 1: Hello World

- ▶ Let's write a simple hello world program write the print function as a compiled Cython function:

```
print('Hello World')
```

- ▶ Let's save this as a helloworld.pyx.

Compiling

- ▶ There are several ways to compile Cython code, including command-line.
- ▶ For more info: https://cython.readthedocs.io/en/latest/src/userguide/source_files_and_compilation.html#compilation
- ▶ We will use a setup.py (build script, similar to a Makefile), example shown below:

```
from setuptools import Extension, setup
from Cython.Build import cythonize

extensions = [Extension("helloworld", ["helloworld.pyx"])]

setup(
    ext_modules = cythonize(extensions)
)
```

- ▶ Setup scripts:
<https://docs.python.org/3.11/distutils/setupscript.html>

- ▶ To compile (command line):
`python setup.py build_ext --inplace`
- ▶ This step creates an intermediate C file and a file with pyd extension.
- ▶ You can use this module now:
`import helloworld`

Cython: C Data Types

- ▶ Cython supports all C data types:
 - ▶ char, short, int, long,
 - ▶ signed and unsigned versions are supported
 - ▶ bool is supported using: bint
 - ▶ pointers are supported

- ▶ We use `cdef` to declare C variables:

```
cdef int i;  
cdef float j;  
cdef int k[10];  
cdef int* p;
```

- ▶ Struct:

```
cdef struct mystruct:  
    int field1;  
    int field2;
```

- ▶ Enum:

```
cdef enum Currencies:  
    USD, EUR, JPY
```

Functions

- ▶ Python functions are defined using `def` (as usual).
- ▶ C functions are defined using `cdef`.
- ▶ C functions can:
 - ▶ take Python objects or arguments declared as C types
 - ▶ return Python objects or arguments declared as C types
- ▶ Arguments of a C or Python function can have C data types.
- ▶ Python and C functions can call each other.
- ▶ Only Python functions can be called from outside the module by interpreted Python code.
- ▶ Any functions that you want to “export” from your Cython module must be declared as Python functions using `def`.

Example 2: Fibonacci

- ▶ Example below uses C types to calculate Fibonacci numbers:

```
def fib(int n):  
    cdef int a = 0  
    cdef int b = 1  
    while (b < n):  
        print(b)  
        a = b  
        b = a + b
```

- ▶ Let's save this to fibs.pyx.

- ▶ We can call this function in a regular Python script:

```
import fib

def print_fib():
    fib.fib(10)
```

- ▶ After compiling the Cython file, we can use it in a regular Python script.

Using Cython in Julia Set Example

- ▶ We don't need the flexibility of dynamic types in Julia Set example – we can use static C types in Cython.
- ▶ Let's start by converting (dynamic) Python types to (static) C types.
- ▶ As you will see, this step alone can give us a big boost in performance in computing-heavy applications.

Cython Julia Set Example: Change 1

- ▶ Let's change `calculate_z()` to use Cython static C types.
(Remember: this is our most expensive function.)

```
cdef unsigned int i, n  
cdef double complex z, c
```

- ▶ Execution times changed from 11 seconds to 0.64 seconds (on my laptop).

Cython Julia Set Example: Change 2

- ▶ We use abs Python function. (Remember: this is another expensive function.)
- ▶ Let's rewrite it without using abs function:

```
while (z.real*z.real + z.imag * z.imag)
```
- ▶ Program runs faster now.

Parallel Work

- ▶ Python runtime is not thread safe – we cannot use multiple threads within a Python process.
- ▶ Python runtime uses a GIL (Global Interpreter Lock) which ensures only one line of python code can be compiled at a time.
- ▶ This means, if even if we use threads only one is thread allowed to run at a given time.
- ▶ In other words, Python multithreading will not improve performance in this case.
- ▶ Cython allows us to use OpenMP to parallelize for loops (bypassing the GIL).

Cython and OpenMP

- ▶ Parallel range (prange) operation is used to run code in parallel.
- ▶ prange are used to parallelize for loops only.
- ▶ We must disable the GIL in parallel loops.
- ▶ We cannot use regular python objects inside parallel loops.

Cython Julia Set Example: Change 3

- ▶ We can write change the for loop to use prange:

```
length_zs = len(zs)
with nogil:
    for i in prange(length_zs)
```

- ▶ We can combine the two lines and write:

```
length_zs = len(zs)
    for i in prange(length_zs, nogil=True):
```

- ▶ We can use static, dynamic or guided policies to schedule OpenMP threads (discussed last week)
<https://docs.microsoft.com/en-us/cpp/parallel/openmp/d-using-the-schedule-clause?view=msvc-160>

- ▶ We cannot use a Python List (output) inside the nogil section.
- ▶ But, we can use a numpy array instead:

```
cdef int[:] output = np.empty(len(zs), dtype=np.int32)
```

- ▶ setup.py has to be modified to use OpenMP.

```
ext_modules = [Extension("julia_calc",  
                        ["julia_calc.pyx"],  
                        include_dirs=[numpy.get_include()],  
                        extra_compile_args=['-fopenmp'],  
                        extra_link_args=['-lgomp'])]
```

- ▶ With Changes 1,2 and 3, runs much faster.

Cython: Features

- ▶ We saw two main features:
 1. Static C types in place of dynamic Python types
 2. Parallel for loops using OpenMP
- ▶ Cython supports many features:
 - ▶ C++ features
 - ▶ C++ Standard Library
 - ▶ ...
- ▶ One needs to know C++ reasonably well to use them in Cython.

Using Multicore Systems: Remarks

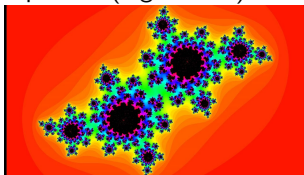
- ▶ Modern computers have many cores.
- ▶ We use threads to utilize them.
- ▶ Learning how to write multithreaded programs is important.
- ▶ We discussed:
 - ▶ How to use `std::thread`
 - ▶ High level constructs (OpenMP, TBB)
 - ▶ Related concepts (low level and high level)
 - ▶ How to use OpenMP in Python applications
- ▶ Applications that can use parallelism:
 - ▶ Black Scholes
 - ▶ Tree pricer
 - ▶ Monte Carlo
 - ▶ Cython examples

Heterogeneous Computing

- ▶ We have seen:
 - ▶ vectorization
 - ▶ multithreading (multicore)
- ▶ Both technologies use the CPU.
- ▶ Next, we will look at how to use GPUs (Graphics Processing Unit) for general purpose computing.

GPU

- ▶ What is a GPU?
- ▶ GPUs are designed to handle display devices:
 - ▶ a picture (e.g. below)⁴ consists of a large number of pixels



- ▶ processing/displaying graphics require a large number of parallel operations
- ▶ GPUs are designed to handle a large number of small operations in parallel

⁴source: Internet

GPGPU Programming

- ▶ GPUs are used in high performance computing – known as GPGPU (General Purpose GPU) computing.
- ▶ Unlike a CPU, a GPU cannot operate standalone.
- ▶ We use it as a co-processor to a CPU.
- ▶ Using different processor types/architectures (e.g. CPU and GPU) together to perform a computation is known as *heterogeneous computing*.
- ▶ CPU and GPU combination is a very popular heterogeneous programming solution:
 - ▶ mature technology
 - ▶ good development tools
 - ▶ low cost

Why Use GPUs

- ▶ Modern CPUs have many cores:
 - ▶ the number of cores is (relatively) small
 - ▶ optimized for sequential serial processing
- ▶ GPUs:
 - ▶ support large number of threads
 - ▶ designed to handle a very large number of small parallel tasks

New Terminology

- ▶ We use some new terms to discuss heterogeneous applications:
 - ▶ host: refers to the CPU and its memory
 - ▶ device: refers to the GPU and its memory
 - ▶ kernel: function we want to execute on the GPU
- ▶ A heterogeneous application has host code and device code:
 - ▶ host code runs on the CPU and uses CPU memory
 - ▶ device code runs on the GPU and uses GPU memory

CUDA (Compute Unified Device Architecture)

CUDA C

- ▶ Language extensions to C to support heterogeneous GPGPU programming.
- ▶ Library solution from NVIDIA Corp.; Provides API and functions to run code on GPUs.
- ▶ Uses a special (nvcc) compiler to compile GPGPU programs:
 - ▶ nvcc compiles device specific code (code that runs on the GPU)
 - ▶ uses a host compiler (e.g. Visual C++/gcc) to compile host code (code that runs on the CPU)
- ▶ Reference: CUDA C++ Programming Guide
<https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>

CUDA: Example 1

- ▶ Let's use a very simple program to illustrate GPU computing using CUDA step-by step.
- ▶ Step 1: Program below is a simple C program to write a message to Console.

- ▶ `<stdio.h>` defines input output functions with standard I/O devices in C.

```
#include <stdio.h>
```

```
void greeting()  
{  
    printf("Hello, World\n");  
}
```

- ▶ Now we can call the function from the `main()`:

```
int main()  
{  
    greeting();  
}
```

- ▶ This program runs on the CPU.

- ▶ Step 2: Change the program to use the GPU using CUDA.
 - ▶ Use the GPU (device) to print the message.
 - ▶ The code that runs on the device (GPU) is known as the *kernel*.
 - ▶ We use the `__global__` qualifier to identify code that runs on the device (more on this later):

```
__global__ void greeting()  
{  
    printf("Hello, World\n");  
}
```

- ▶ A kernel is executed by CUDA threads on the device.

Launching a Kernel Function

- ▶ We launch the kernel function from the host:

```
int main()
{
    gretting<<<1, 10>>>();

    cudaDeviceSynchronize();
}
```

- ▶ CUDA manages kernels on GPU threads using an *execution configuration*.
 - ▶ Uses triple arrow syntax:
gretting<<<1, 10>>>();
 - ▶ Specifies how CUDA threads are organized.
- ▶ In this case we use 10 threads (more on this later).
- ▶ cudaDeviceSynchronize() waits for GPU threads to finish.

Example: Hello World

- ▶ We write CUDA programs using the .cu extension.⁵
- ▶ E.g. you can save the following program to hello.cu:

```
#include <stdio.h>

__global__ void greeting()
{
    printf("Hello, World\n");
}

int main()
{
    greeting<<<1, 10>>>();

    cudaDeviceSynchronize();
}
```

⁵Other file types/extensions are possible in a CUDA project but we limit our use to .cu only.

Build/Run the Program

- ▶ We need CUDA libraries and build tools to build this program.
- ▶ We need a NVIDIA GPU to run this code:
- ▶ To request a GPU compute node:

```
sinteractive --partition=gpu --gres=gpu:1 --time=0:30:00 --account=finm32950
```

- ▶ Load CUDA modules:⁶

```
module load cuda/11.7
```

- ▶ Important: Getting a cuda node may require a few tries depending on the day of the week and time of the day. Generally you should have better success after regular work hours and on weekends.

⁶We use 11.7 CUDA Toolkit version in this course.

- ▶ We use the NVIDIA C Compiler (nvcc) to build the program:

```
nvcc hello.cu -o hello
```

- ▶ Options:

- ▶ -o exe-name

- ▶ To run:

```
./hello
```

Further Studies: Installing CUDA Toolkit

- ▶ Installing CUDA toolkit on Windows/Linux is easy.
`https://developer.nvidia.com/cuda-downloads`
- ▶ But it not necessary for the purposes of this course. Everyone is expected to use midway.
- ▶ Note to Mac users: `https://developer.nvidia.com/nvidia-cuda-toolkit-developer-tools-mac-hosts`

Execution Configuration

- ▶ This execution configuration uses one GPU thread:
`greeting<<<1, 1>>>();`
- ▶ You will see one greeting message.
- ▶ We can change the execution configuration to use more threads.
- ▶ To use 10 GPU threads:
`greeting<<<1, 10>>>();`
- ▶ To use 1000 GPU threads:
`greeting<<<1, 1000>>>();`

`greeting<<<10, 100>>>();`
- ▶ These observations illustrate:
 1. We write a CUDA kernel as a sequential program.
 2. A kernel is executed by CUDA threads.
 3. Execution configuration specifies the number of threads.
- ▶ Execution configuration is important to use CUDA efficiently (more on this later).

CUDA: New Keywords

- ▶ CUDA extends C language with syntax to define kernels.
- ▶ CUDA C defines following keywords:
 - ▶ `__global__`:
 - ▶ used to identify the kernel
 - ▶ callable from host or device
 - ▶ executed on device
 - ▶ must return void
 - ▶ `__device__`:
 - ▶ callable from device only
 - ▶ executed on device
 - ▶ `__host__`:
 - ▶ callable from host only
 - ▶ executed on host only
 - ▶ not required to use this qualifier

Data Parallelism and SIMT

- ▶ GPGPU is an attractive solution for data parallel problems.
- ▶ Data parallelism: execute the same operation on different data in parallel.
- ▶ Executing a kernel function in parallel using many GPU threads is known as SIMT (Single Instruction Multiple Thread).
- ▶ We will examine SIMT using a better example next.

Example: Vector Addition

- Suppose we want to add two arrays (vectors):

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} a_1 + b_1 \\ a_2 + b_2 \\ a_3 + b_3 \end{bmatrix}$$

```
void add(float* a, float* b, float* c, int N)
{
    for (int i=0; i<N; ++i)
    {
        c[i] = a[i] + b[i];
    }
}
```

- As we discussed earlier, this is naturally parallel problem.

Vector Addition on the CPU

- ▶ First, let's write a C program to run this program on the CPU:

```
int main()
{
    int N = 1024;
```

- ▶ Declare 3 arrays:

```
float* h_a, *h_b, *h_c;
```

- ▶ Allocate memory:⁷

```
h_a = (float *)malloc(N*sizeof(float));
h_b = (float *)malloc(N*sizeof(float));
h_c = (float *)malloc(N*sizeof(float));
```

- ▶ We use the malloc function to allocate a given size of memory.

<http://www.cplusplus.com/reference/cstdlib/malloc/>

⁷large arrays must be created on the free-store.

- ▶ Initialize data:

```
for (int i=0; i<N; ++i)
{
    h_a[i] = ...
    h_b[i] = ...
}
```

- ▶ Use the function:

```
add(a, b, c, N);
```

- ▶ Use the results

- ▶ Free memory:

```
free(h_a);
free(h_b);
free(h_c);
}
```

- ▶ We use the free function to deallocate a memory block.

<http://www.cplusplus.com/reference/cstdlib/free/>

Vector Addition on the GPU

- ▶ Recall:

- ▶ We write the kernel as a sequential program.
- ▶ Data parallel: execute the same kernel function using CUDA threads.

Suppose, we want to use one thread to compute/add one element. We need 1024 threads to add the two arrays.

thread 0: $c[0] = a[0] + b[0]$

thread 1: $c[1] = a[1] + b[1]$

...

thread 1023: $c[1023] = a[1023] + b[1023]$

- ▶ We can write a kernel as:

```
__global__ void add(float* a, float* b, float* c, int N)
{
    c[i] = a[i] + b[i];
}
```

- ▶ NOTE: this is our first try, and kernel function above is NOT complete.

Execution Configuration

- ▶ We must create 1024 threads to compute each element in the array ($N=1024$) in this case.
- ▶ Earlier, we saw that the execution configuration specifies the number of threads (i.e. $N = X * Y$):
`add<<<X, Y>>>(a, b, c, N);`
- ▶ Questions:
 1. What are X and Y ?
 2. How do we assign threads to each array index?

Grids, Blocks and Threads

- ▶ To answer: we need to discuss how threads are organized in the kernel.
- ▶ All threads in a kernel launch is known as a grid.
- ▶ A grid is made up of (thread) blocks – a block is a group of threads.⁸

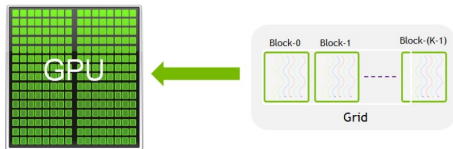


Figure 2. CUDA kernels are subdivided into blocks.

- ▶ Threads in a thread block can cooperate with each other.
- ▶ Threads in different blocks cannot cooperate with each other.
- ▶ Thread blocks are executed independently in any order, in parallel or serially.

⁸Image source: <https://developer.nvidia.com/blog/cuda-refresher-cuda-programming-model/>

Block and Thread IDs

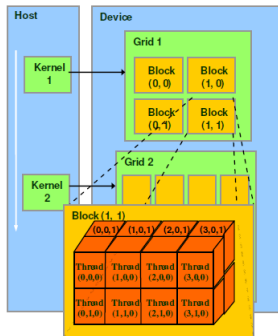
- ▶ CUDA runtime assigns each thread two values:
 - ▶ blockIdx: initialized with the block index within a grid
 - ▶ threadIdx: initialized with the thread index within a block
- ▶ The blockIdx and threadIdx together identifies a thread uniquely.

`threadID = threadIdx + blockIdx * blockDim`

- ▶ threadID above is not exactly correct because ... (next slide)

Block and Thread IDs

- ▶ The blockIdx and threadIdx are 3 dimensional coordinates:
 - ▶ Each coordinate is given by x, y, and z.
 - ▶ A block index has: blockIdx.x, blockIdx.y, blockIdx.z
 - ▶ A thread index has: threadIdx.x, threadIdx.y, threadIdx.z
 - ▶ Can be used to form 1-D, 2-D, 3-D thread blocks to provide a natural way to invoke computation across the elements in a domain such as a vector, matrix, or volume.



- ▶ We will limit our discussion to 1-D domains only.

Block and Thread Dimensions

- ▶ An execution configuration uses two arguments:
 - ▶ 1st: number of blocks in the grid
 - ▶ 2nd: number of threads in a block
- ▶ E.g.: Suppose we want to create 4 blocks with 16 threads in each block.
- ▶ For 1-D domain, we can use an execution configuration as:
`myKernel<<<4, 16>>>();`

- ▶ We need to use `dim3` type for 2-D and 3-D domains (not necessary for 1-D):

```
dim3 dimGrid(4, 1, 1);  
dim3 dimBlock(16, 1, 1);  
myKernel<<<dimGrid, dimBlock>>>();
```

- ▶ The figure above (previous slide) shows:

```
dim3 dimGrid(2, 2, 1);  
dim3 dimBlock(4, 2, 2);  
Kernel1<<<dimGrid, dimBlock>>>();
```

- ▶ We have several options to get N parallel threads:
 - ▶ `<<<N, 1>>>`
 - ▶ `<<<1, N>>>`
 - ▶ `<<<M,  $\frac{N}{M}$ >>>`
- ▶ The maximum number of threads in the block is limited to 1024.
- ▶ Tech spec: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#features-and-technical-specifications>
- ▶ Compute capabilities: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#compute-capabilities>

- ▶ Usually, we use a combination of blocks and threads.
- ▶ The multiprocessor schedules and executes threads in groups of 32 parallel threads called *warps*.
- ▶ General practice: block size is a multiple of 32 (e.g. 128, 256).
- ▶ Important: Performance depends on the application/problem (this is key point we highlight in this course):
 - ▶ No rule works for all apps.
 - ▶ Experiment, measure time and profile.

Thread IDs

- ▶ Suppose we use X blocks with Y threads-per-block.
- ▶ We can use `threadIdx` and `blockIdx` to find a unique thread index for each thread as:⁹

```
int threadID = threadIdx.x + blockIdx.x * blockDim.x;
```

- ▶ Threads-per-block is a predefined (within a kernel) number, given by `blockDim`.
- ▶ For example, for <<<4, 8>>> execution configuration:

<code>threadIdx.x</code>	<code>blockIdx.x</code>	<code>blockDim.x</code>	<code>threadID</code>
0	0	8	0
1	0	8	1
2	0	8	2
..	..	..	..
0	1	8	8
1	1	8	9

⁹1-D only

Example: Vector Addition Cont ..

- ▶ Let's use 16 blocks with 64 threads per block.

```
add<<<16, 64>>>(a, b, c, N);
```

- ▶ Thread index goes from 0 to 1023.
- ▶ We can now assign a thread to each array index using the thread index:

```
__global__ void add(float* a, float* b, float* c, int N)
{
    int threadId = threadIdx.x + blockIdx.x * blockDim.x;

    c[threadId] = a[threadId] + b[threadId];
}
```

- ▶ Kernel above assumes we have sufficient threads to process each element in the array in one pass.
- ▶ What if N is larger than the number of threads, e.g. N=2048?
- ▶ We can modify the kernel slightly for this case (N=2048; threads in the grid=1024):

```
thread 0:          c[0] = a[0] + b[0]
thread 1:          c[1] = a[1] + b[1]
...
thread 1023:       c[1023] = a[1023] + b[1023]

thread 0:          c[1024] = a[1024] + b[1024]
thread 1:          c[1025] = a[1025] + b[1025]
...
thread 1023:       c[2047] = a[2047] + b[2047]
```

► Modified kernel:

```
__global__ void add(float* a, float* b, float* c, int N)
{
    int threadId = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;
    for (int idx=threadId; idx<N; idx += stride)
    {
        c[idx] = a[idx] + b[idx];
    }
}
```

- Where $\text{stride} = \text{total number of threads in the grid}$, given by $\text{blockDim.x} * \text{gridDim.x}$
- This form is known as a *grid-stride loop*.
- This form has several advantages:
1. Handles $N > \text{total-number-of-threads in the grid}$.
 2. Scalability: we can use the same kernel/solution to handler larger N . I.e., grid side is decoupled from the data size.
 3. Thread reuse: using a thread more than one computation increases efficiency.

Memory Structure

- ▶ CPU and GPU have their own memory.
- ▶ A CUDA program has the following structure:

```
int main()
{
    1. allocate and initialize CPU memory

    2. allocate GPU memory

    3. copy data (inputs) to GPU memory (from CPU memory)

    4. launch CUDA kernel; compute;
       generate results on GPU memory

    5. copy data (results) to CPU memory (from GPU memory)

    6. deallocate GPU memory
    7. deallocate CPU memory
}
```

- ▶ We know how to do steps: 1, 4, 7
- ▶ We don't know how to do steps: 2, 3, 5, 6

CUDA C Memory Operations

- ▶ CUDA C supports functions to:
 - ▶ Allocate/deallocate memory on the GPU
 - ▶ Copy (input) data from CPU memory to GPU memory
 - ▶ Copy (results) data from GPU memory to CPU memory
- ▶ These functions model standard C memory functions.

- ▶ To allocate memory on the GPU, we use `cudaMalloc` function.
- ▶ To deallocate memory on the GPU, we use `cudaFree` function.
- ▶ To copy memory, we use `cudaMemcpy` function:
 - ▶ use `cudaMemcpyKind` argument shows the direction
 - ▶ copy from host to device: `cudaMemcpyHostToDevice`
 - ▶ copy from device to host: `cudaMemcpyDeviceToHost`
- ▶ Ref: http://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__MEMORY.html#axzz4mGltmj9b

Recommendation: Variable Naming Convention

- ▶ Now we're dealing with host and device data (memory/variables).
- ▶ Identifying them clearly is important.
- ▶ Following naming convention is widely used to identify the host and device memory easily:
 - ▶ Host memory (host data) is identified using `h_`
`float* h_a;`
 - ▶ Device memory (device data) is identified using `d_`
`float* d_a;`

Example 2: CPU Memory Operations

- ▶ Now we can complete the vector addition:

```
int main()
{
    int N = 1024;
```

- ▶ Allocate host memory:

```
    float *h_a, *h_b, *h_c;
    h_a = (float *)malloc(N*sizeof(float));
    h_b = (float *)malloc(N*sizeof(float));
    h_c = (float *)malloc(N*sizeof(float));
```

- ▶ Initialize data:

```
    for (int i=0; i<N; ++i)
    {
        h_a[i] = i; //for illustration
        h_b[i] = i;
        h_c[i] = 0;
    }
```

Example 2: GPU Memory Operations

- ▶ Allocate memory on the device:

```
float* d_a, *d_b, *d_c;
```

```
cudaMalloc((float**)&d_a, N*sizeof(float));  
cudaMalloc((float**)&d_b, N*sizeof(float));  
cudaMalloc((float**)&d_c, N*sizeof(float));
```

- ▶ Copy data to device:

```
cudaMemcpy(d_a, h_a, N*sizeof(float),  
           cudaMemcpyHostToDevice);  
  
cudaMemcpy(d_b, h_b, N*sizeof(float),  
           cudaMemcpyHostToDevice);
```

- ▶ Memory transfers above are synchronous (blocking) calls.
- ▶ Why pointer to pointer (double pointer):

[https://stackoverflow.com/questions/7989039/
use-of-cudamalloc-why-the-double-pointer](https://stackoverflow.com/questions/7989039/use-of-cudamalloc-why-the-double-pointer)

- ▶ Launch kernel:

```
add<<<16, 64>>>(d_a, d_b, d_c, N);
```

- ▶ Kernel launch is an asynchronous (not a blocking) call. We wait for CUDA threads to run:

```
cudaDeviceSynchronize();
```

- ▶ Copy data back to host. This operation won't start until kernel execution has completed.

```
    cudaMemcpy(h_c, d_c, N*sizeof(float),  
               cudaMemcpyDeviceToHost);
```

- ▶ Now we have the results on the host.
- ▶ Free device memory:

```
    cudaFree(d_a);  
    cudaFree(d_b);  
    cudaFree(d_c);
```

- ▶ Free host memory:

```
    free(h_a);  
    free(h_b);  
    free(h_c);
```


- ▶ To build (on midway3):

```
module load cuda/11.7
```

```
nvcc add_vector.cu -o add_vector
```

Error Handling

- ▶ CUDA functions use error codes to return information about errors.
- ▶ Error codes have the type `cudaError_t`.
- ▶ If an operation succeeds, error code is set to `cudaSuccess`.
- ▶ When we call CUDA functions we should always make sure they succeed:

```
cudaError_t errcode = cudaMalloc(/*...*/);  
if (errcode != cudaSuccess)  
{  
    //handle error  
}
```

- ▶ CUDA provides utility functions to handle error codes:
 - ▶ `cudaGetErrorString` returns a string to describe the error
 - ▶ `cudaGetLastError` returns the last error
- ▶ See Demo (vector addition 3) for an example.